

hardware-hunter

Product Requirements Document — v1

Author: ifuensan

Date: 2026-05-10

Project type: cli_tool · self-hosted agent

Domain: general (marketplace agent / homelab tooling)

Complexity: high

Release mode: phased

License: MIT

Repository: github.com/ifuensan/hardware-hunter

(c3) scope contract — do not relitigate. Personal homelab tool. Wallapop + eBay.es only for v1; multi-marketplace deferred. Arbitrage explicitly out of scope and structurally prevented.

Executive Summary

hardware-hunter is a self-hosted, MIT-licensed agent that watches Wallapop and eBay.es continuously against a YAML wishlist of specific homelab parts (HDDs, RAM) and surfaces matches in Telegram — including parts hidden inside larger listings (a WD Red inside a “NAS DS220 con discos”) that saved searches structurally cannot see. With opt-in Phase 2 enabled per wishlist entry, an explicit Telegram tap completes the purchase via the marketplace’s protected payment rail.

Primary user. ifuensan (Spanish homelabber) personally; the OSS release is a side benefit for other Spanish homelabbers, not a go-to-market. No team, no funding, no resale.

Problem. Manual scanning of Wallapop and eBay.es costs a homelabber roughly 20 minutes a day of refreshing the same searches; good listings vanish in about four hours; saved searches cannot reason per-SKU about price ceilings, cannot verify a listing matches its claim, and never see hidden-component listings at all. The DRAM/NAND price spike (DDR4 32GB doubled Oct 2025 → Q1 2026; shortages projected through Q4 2027 per Tom’s Hardware and IDC) makes the Spanish second-hand market the rational sourcing channel — and the time tax peaks alongside it.

Future state. ifuensan no longer refreshes Wallapop. The wishlist YAML and Telegram replace the manual hunt entirely; Phase 2 closes the “I was in a meeting” gap.

Scope contract — (c3), do not relitigate. Personal homelab tool. Wallapop + eBay.es only for v1; multi-marketplace deferred. Arbitrage is explicitly out of scope and structurally prevented (no `expected_resale_value` field; LLM prompt is wishlist-anchored, not arbitrage-anchored). Arbitrage forks belong in a separate repo under a different name.

What Makes This Special

Four differentiators, none of which the existing OSS Wallapop bots (wallabot, Walla-Bot, daverator, nadiamoe, Tatuck) or Wallapop’s own saved searches combine:

1. **Specific-model wishlist with per-SKU price ceilings.** YAML names exact references (`manufacturer`, `model`, `ref`) and two ceilings — `max_price_solo` (standalone) and `max_price_in_device` (hidden inside a wrapper). Alerts fire on the part the user actually wants at the price they’d actually pay, not on keyword matches.
2. **Container detection.** A listing’s title/description/photos are evaluated for whether the wanted SKU is inside a NAS, mini-PC, or workstation. No competitor does this; saved searches catch 0% of hidden-component listings by definition.
3. **Phase 2: opt-in autonomous purchase via the marketplace’s protected payment rail** (Wallapop Pay, eBay checkout — never Bizum or transferencia), gated by a non-bypassable Telegram tap. Per-entry toggle, per-entry max price, per-entry confidence threshold. No fully autonomous mode by design.
4. **eBay.es coverage alongside Wallapop.** eBay.es regularly surfaces server-class hardware that never appears on Wallapop; eBay’s adapter is structurally independent, so a Wallapop break leaves the system half-degraded, not dead.

Core insight. The wishlist YAML makes “what to buy” deterministic — the LLM only verifies whether a listing matches a wishlist entry, never picks. This single design choice does triple duty:

- Sidesteps the ACES-style biases (position, sponsored, open-ended pricing) that affect VLM shopping agents.
- Structurally prevents arbitrage drift — schema has no `expected_resale_value`, prompt is wishlist-anchored, no off-wishlist alerts. Drift requires a deliberate change visible in code review.
- Reduces the LLM failure surface to one verifiable question per listing, orthogonal to the structural buy-flow guardrails (cross-source price reconciliation, receipt-vs-alert diff, daily synthetic smoke test) that defend against silent Phase 2 misbehavior.

Project Classification

Field	Value
Project Type	<code>cli_tool</code> — self-hosted agent, docker-compose install, YAML/env config, Telegram as the user surface (no built-in GUI)
Domain	<code>general</code> (sub-domain: marketplace agent / homelab tooling)
Complexity	High — driven by Phase 2 autonomous purchase, two-path Wallapop adapter under anti-bot pressure, silent-failure-mode design, LLM evaluation with confidence, unproven container-detection accuracy
Project Context	Greenfield — fresh repository at <code>github.com/ifuensan/hardware-hunter</code> ; no prior code or docs
License / repo	MIT, personal GitHub (<code>ifuensan/hardware-hunter</code>)
Stack (locked, not for PRD debate)	Hermes Agent v0.13.0 (Nous Research, MIT) + TinyFish via MCP; runs on owned HPE DL160 Gen10 in Valencia colo; Telegram for alerts and approvals; LLM (Gemini Flash assumed) for listing evaluation

Success Criteria

User Success

The user (ifuensan) achieves success when:

- **Time-tax elimination.** ~20 min/day of manual Wallapop/eBay.es scanning replaced by passive Telegram alerts. By month 1 of Phase 1, manual scanning is ≤ 2 min/day (spot-check only).
- **Deal capture against the 4-hour half-life.** Within the first 30 days of Phase 1, at least one alert fires on a listing the user would have missed manually (off-hours, container, or four-hour-window).

- **Container detection is not vapor.** Within the first 90 days, ≥ 1 container-detection alert (HDD-in-NAS or RAM-in-mini-PC) the user confirms as a true positive on tap. Zero across 90 days = headline differentiator is broken; rethink before promoting publicly.
- **Phase 2 trust acquired.** After 4–8 weeks of Phase 1 reading, the user enables Phase 2 for at least one wishlist entry; first Phase 2 purchase completes successfully (item arrives, matches the alert, no buyer-protection dispute).
- **Aha! moment.** First Telegram alert that surfaces a match the user would have bought manually anyway — while doing something else.

Project Sustainability (in lieu of “Business Success”)

This is OSS scratch-own-itch; success $\neq$ commercial metrics. Sustainability is the right frame:

- **Personal-utility holds.** ifuensan keeps using hardware-hunter for own homelab parts for 3+ consecutive months (inverse of the personal-use walk-away trigger).
- **Maintenance cost contained.** Steady-state ≤ 8 h/month after the first 6 months. Any rolling 3-month average > 20 h/month invokes the sustained-burden walk-away.
- **Marketplace breaks absorbable.** Expected 2–4 breaks/year, each absorbed in ≤ 30 h. Three consecutive failed patch attempts on a single break = technical-debt walk-away trigger.
- **Cost ceiling held.** Running cost stays at €0 on existing homelab hardware (or $\leq$ €10/month worst-case if TinyFish free tier disappears).
- **Graceful off-ramp documented.** If any walk-away trigger fires: final commit pinning deps, README addendum dating the unmaintained state, repo archived with pointer to forks. Not silent abandonment.
- **OSS adoption is explicitly NOT a success criterion** (per (c3)). External users are a side effect; community PRs welcome but unbudgeted.

Technical Success — v1 launch-blocker bar

- **Adapter discipline enforced.** `PageFetcher` / `BrowserSession` (or equivalent) interfaces wrap Hermes and TinyFish. Direct imports from business logic = launch blocker. Verified via CI lint or code-review checklist.
- **Two-path Wallapop adapter operational.** Unofficial API primary + TinyFish Search/Fetch fallback. Either path alone carries Phase 1.
- **eBay.es adapter operational.** Official eBay API; structurally independent from Wallapop’s adapter.
- **Phase 1 stabilization gate.** 4–8 weeks of Phase 1 running before Phase 2 enables for any wishlist entry.
- **Phase 2 fail-closed.** Purchase refused unless 100% of expected UI elements present.
- **Per-purchase circuit breaker.** N consecutive Phase 2 failures auto-disables autonomous mode globally and Telegrams the user.
- **Cross-source price reconciliation at buy time.** Re-fetch via the alternate path before checkout; tolerance breach fails closed.
- **Receipt-vs-alert reconciliation.** Mismatch raises high-priority Telegram alert and auto-disables Phase 2 (scope of auto-disable — global vs per-entry — captured as an Open Question for the

PRD body).

- **Daily Phase 2 synthetic smoke test.** Drift auto-disables Phase 2.
- **Append-only SQLite audit log per Phase 2 purchase.** Alert snapshot + tap event + marketplace transaction (with photo perceptual hashes).
- **No remote logging, no telemetry.** All data local; user = data controller.

Measurable Outcomes

Metric	Target	Triggered remediation
Manual scanning post Phase 1 month 1	≤ 2 min/day	Phase 1 UX is broken; investigate alert reliability
Container-detection true positives, first 90 days	≥ 1 user-confirmed	Differentiator is vapor; rethink before public framing
Phase 2 enabled on ≥ 1 entry by week 12	Yes	Trust never acquired; reassess Phase 2 worth shipping
Phase 2 silent mispurchases, first 90 days	0	Reconciliation stack insufficient; pause Phase 2
Maintenance hours, rolling 3-month avg	≤ 20 h/month	Sustained-burden walk-away
Marketplace-break recovery	≤ 30 h, ≤ 3 attempts per break	Technical-debt walk-away
Running cost	$\leq \text{€}10/\text{month}$ worst-case	Cost-creep audit

Product Scope

MVP — Phase 1 (alerts only)

The minimum that proves the concept: continuous Wallapop + eBay.es watch against the wishlist YAML, Telegram alerts with confidence levels and one-line LLM takes, no purchase action.

In MVP:

- Wishlist YAML schema: `manufacturer`, `model`, `ref`, `max_price_solo`, `max_price_in_device`, `type` (hdd|ram), `keywords[]`, `container_keywords[]`. Up to ~100 entries.
- Wallapop two-path adapter (unofficial API + TinyFish Search/Fetch fallback) behind `PageFetcher` / `BrowserSession` interfaces.
- eBay.es official-API adapter.
- Wishlist-anchored LLM evaluation with confidence level; container detection logic.
- Telegram alerts with photo, price, seller location, one-line LLM take, link; skip/snooze buttons.

- SQLite seen-listings dedup; cron-driven polling.
- Wallapop long-lived sessions with manual re-auth (no silent automated re-login).
- Single `docker-compose` install; `.env.example`; README with example wishlist entries for common HDD/RAM models.
- **ROADMAP.md and CONTRIBUTING.md as launch artifacts** (not nice-to-haves) — `CONTRIBUTING.md` carries the explicit “no arbitrage PRs” rule; `ROADMAP.md` names future-multi-marketplace, future-arbitrage-as-separate-repo, and “C&D-induced sunset” as documented end-state.
- Repository hygiene: no Wallapop trademarks, logos, or proprietary terms in titles, package names, or domain.

Out of MVP (deferred): Phase 2 buy flow, accuracy dashboard, multi-marketplace, web UI/dashboard, polish.

Growth Features (Post-MVP) — Phase 2 + supporting

Ships ~4–8 weeks after Phase 1 stabilizes for the user:

- Phase 2 autonomous purchase with the full guardrail stack (per-entry toggle, per-entry max price, per-entry confidence threshold, fail-closed UI checks, per-purchase circuit breaker).
- Telegram Buy/Skip buttons per alert (only when Phase 2 is enabled for that entry).
- Wallapop chat → Wallapop Pay flow via TinyFish Browser; eBay.es Cómpralo-ya checkout.
- Cross-source price reconciliation at buy time; receipt-vs-alert reconciliation with auto-disable on mismatch; daily Phase 2 synthetic smoke test.
- Append-only SQLite audit log per Phase 2 purchase.
- **Accuracy dashboard for hidden-component detection.** Per PRFAQ verdict, treated as launch-week priority not fast-follow; placed in Growth only because the data to build it accumulates during Phase 1 use.
- **Empirical LLM bias audit** (esp. Castilian vs Catalan/regional Spanish/Basque language register) before Phase 2 is enabled for any user beyond ifuensan.

Vision (Future, separate research / not v1)

- Multi-marketplace expansion beyond Wallapop + eBay.es (“much later, after validating Wallapop+eBay.es first”); documented in ROADMAP as a future research direction.
- Arbitrage / flipping framing — **explicitly out of scope for hardware-hunter forever**; ROADMAP points to a separate-repo path under a different name.
- Possible publication on agentskills.io for Hermes ecosystem visibility.
- “C&D-induced sunset” as a documented possible end state — not a goal, a planned graceful exit per the legal posture.

User Journeys

The user surface is narrow by design ((c3) personal-use): one human (ifuensan) wearing two hats — homelabber and operator — plus an opt-in fork runner from the OSS side-effect audience. There is no API consumer, no admin separate from the operator, no support staff.

Journey 1 — Phase 1 happy path: the aha!

Persona. ifuensan, Spanish homelabber, runs a Synology RS818+ NAS in a closet in Valencia. Has been watching the second-hand market for a WD Red 4TB to round out his array. DDR4/HDD prices are climbing weekly.

Opening scene. Tuesday, 16:42. He's at his day job. His wishlist YAML has 18 entries; hardware-hunter has been running on the homelab DL160 for 11 days; he has not opened Wallapop today.

Rising action. His phone buzzes. Telegram:

“📦 **WD Red 4TB · 48€ · Valencia · seller online · LLM take: photos show a real WD Red, not a green; serial visible. Confidence: high. [Ver listing]**”

The listing is 9 minutes old. Saved searches haven't pinged yet because his Wallapop saved-search ranges weren't tight enough.

Climax. He taps the link, opens Wallapop, messages the seller, agrees within minutes. Total elapsed: 90 s alert-to-message.

Resolution. Pays via Wallapop Pay. Drive arrives Friday. He never opened Wallapop manually. He has not lost his lunch break to refreshing.

Capabilities revealed. Marketplace polling at human-volume cadence; wishlist-anchored evaluation; per-SKU `max_price_solo`; one-line LLM take with confidence; Telegram alert with photo + price + seller location + deep link; cron scheduling; SQLite seen-listings dedup so this listing won't fire twice.

Journey 2 — Phase 2 happy path: trust acquired

Persona. Same ifuensan, six weeks into Phase 1. He has watched ~40 alerts fire, audited the LLM evaluations, manually purchased four times. This morning he flipped Phase 2 to `true` for two specific entries — WD Red 4TB and HGST He10 14TB — leaving the other 16 entries alert-only.

Opening scene. Wednesday, 11:17. He's in a 1:1 with his manager.

Rising action. Phone buzzes:

“🟢 **HGST He10 14TB · 60€ (max_solo: 75€) · Valencia · confidence: high · real He10, SMART screenshot included · [✓ Buy] [✗ Skip] [👁 Ver]**”

He glances at the photo, taps **Buy** without leaving the meeting.

Climax. Behind the tap: the agent re-fetches the same listing through the *other* path (TinyFish Browser, since the API surfaced this one) — cross-source price reconciliation: 60€ ↔ 60€, within tolerance. Fail-closed UI element check passes. Wallapop Pay checkout proceeds. ~22 seconds end-to-end.

Resolution.

" **Comprado. 60.00€** via Wallapop Pay · Receipt: WAL-7Q4-XYZ · [screenshot]"

Local SQLite audit log records: alert snapshot (with photo perceptual hash), tap event (Telegram message ID + button + timestamp), marketplace transaction (receipt ID + price paid + screenshot). Drive arrives Friday.

Capabilities revealed. Per-entry Phase 2 toggle; per-entry `max_price_solo` ceiling; confidence threshold gating; fail-closed UI element checks; cross-source price reconciliation at buy time; Wallapop Pay flow via TinyFish Browser; append-only audit log per Phase 2 purchase; receipt screenshot capture.

Journey 3 — Edge case: silent-failure caught (the Q9 scenario)

Persona. Same ifuensan, three months in, Phase 2 enabled on three entries. He's stopped reading every alert because they're routine.

Opening scene. Thursday morning. Wallapop has quietly shipped a UI change — the listing detail page now formats the price `1.234,56 €` as `1234.56 €`. The unofficial-API response format is unchanged; only the HTML detail-page parser is affected.

Rising action. The agent's HTML parser silently misreads. A WD Red 4TB priced at 53€ is parsed as 0.53€. The **daily Phase 2 synthetic smoke test** runs at 06:00 against a synthetic listing whose raw text uses dot — it passes. Drift goes undetected by smoke test alone.

A real listing fires. The agent surfaces alert "0.53€". Before any Phase 2 purchase can complete, the **cross-source price reconciliation at buy time** independently re-fetches via the alternate path: API path returns 53€, HTML parser returns 0.53€. Disagreement exceeds tolerance.

Climax. Phase 2 fails closed. High-priority Telegram:

" **Phase 2 auto-disabled (global).** Cross-source price reconciliation tripped: API 53.00 vs HTML parser 0.53. Listing: [link]. Investigate parser."

Resolution. ifuensan reads the audit log on the homelab box, traces drift to the format change, patches the parser, adds comma-formatted fixtures to the smoke-test regression set, re-runs smoke tests green, manually re-enables Phase 2. Total: ~45 minutes. **No money lost.** The reconciliation stack did exactly what the PRFAQ Q9 mitigation promised.

Capabilities revealed. Daily synthetic Phase 2 smoke test; cross-source price reconciliation at buy time as an *independent* second defense (the smoke test missed the bug because its synthetic data didn't cover commas; reconciliation caught it on real data); global Phase 2 auto-disable on

reconciliation failure (TBD whether per-entry instead — see Open Questions); audit log inspection workflow; manual Phase 2 re-enable after green smoke test.

Journey 4 — Operator hat: Wallapop re-auth + wishlist update

Persona. ifuensan-as-operator, Saturday-morning maintenance window. Same person, different mode: he's not chasing deals, he's keeping the agent healthy.

Opening scene. 09:30. Logs show 6 hours of `Wallapop unofficial API: 401 Unauthorized` — session cookie expired. Phase 1 was degraded on Wallapop overnight (the TinyFish Search/Fetch fallback path carried partial coverage), but eBay.es ran clean (independent adapter). Earlier Telegram:

" Wallapop session expired. Manual re-auth required (no silent re-login). Run `hardware-hunter login wallapop`."

Rising action. He SSHes to the DL160, runs `hardware-hunter login wallapop`. The agent spins up a TinyFish Browser session, opens Wallapop's login page in a stealth Chromium, walks him through entering credentials + 2FA, persists the cookie file with restrictive perms. While he's there, he edits `wishlist.yaml` to add a Kingston KVR32N22D8/32 DDR4-3200 32GB kit he's been eyeing.

Climax. He runs `hardware-hunter validate-wishlist` — schema lints clean. He runs `hardware-hunter test-search "Kingston 32GB DDR4 3200"` — the agent surfaces three current listings as a read-only dry run, applying the wishlist-anchored prompt against each so he can sanity-check whether the LLM evaluation matches his judgment.

Resolution. He restarts the stack: `docker-compose down && docker-compose up -d`. Phase 1 backfills overnight; the new Kingston entry enters rotation. Total time: 12 minutes.

Capabilities revealed. Wallapop manual re-auth UX (long-lived session with no silent automated re-login); wishlist YAML schema validator (CLI); dry-run search command; cookie file with restrictive permissions; docker-compose lifecycle as the supported install model; eBay.es independence as a "half-degraded > dead" property.

Journey 5 — OSS contributor: fork runner

Persona. Marc, another Spanish homelabber. Runs a small Mini PC at home with Proxmox and a couple of containers. Spotted hardware-hunter on a Spanish dev forum. Wants a Kingston 16GB DDR4-2666 SO-DIMM under €25.

Opening scene. Sunday afternoon. He's read the README and the legal disclaimer. He spins up a fresh Telegram bot via BotFather, generates a Wallapop secondary account he's "willing to lose" (per the customer-FAQ recommendation), clones the repo.

Rising action. Copies `.env.example` → `.env`, fills in Telegram bot token + Wallapop credentials + eBay.es app credentials. Edits the example `wishlist.yaml` down to one entry (his Kingston module), tunes `max_price_solo` to 25€. Runs `docker-compose up -d`. Phase 2 is **disabled by default** — the README is explicit.

Climax. Two days later: "Kingston KVR26N19S8/16 · 22€ · Madrid · confidence: medium · *photos show package, label visible* · [link]". He taps the link, buys manually via Wallapop Pay.

Resolution. He stars the repo. Considers opening a PR with three additional Castilian-friendly wishlist examples for the README — the `CONTRIBUTING.md` lists wishlist contributions as one of the three explicit invitation categories. He never enables Phase 2; alerts are enough for him. (This is the (c3)-aligned fork-user posture: hardware-hunter saved him time, he didn't need the autonomous-purchase tier.)

Capabilities revealed. README with realistic example wishlists; `.env.example` covering all needed credentials; single `docker-compose up` install; Phase 2 disabled-by-default for safety; `CONTRIBUTING.md` with three invitation categories (wishlist examples, prompt improvements, Wallapop selector patches); legal disclaimer + secondary-account recommendation in README.

Journey Requirements Summary

Capability area	Driven by journeys	Notes
Marketplace polling (Wallapop two-path + eBay.es)	1, 2, 4, 5	Cron-driven, human-volume rates, eBay.es independent
Wishlist-anchored LLM evaluation w/ confidence	1, 2, 5	One-line take returned per match
Container detection	(adjacent — implied baseline; full demo in a future story)	First true-positive alert is a launch-week metric
Telegram alerts (rich format + skip/snooze)	1, 2, 3, 4	Phase 1 buttons: skip / snooze; Phase 2 adds Buy / Skip
Per-entry Phase 2 toggle / max price / confidence threshold	2, 3	Phase 2 default <code>false</code> (Journey 5)
Fail-closed UI element checks + per-purchase circuit breaker	2, 3	Auto-disables on N consecutive failures
Cross-source price reconciliation at buy time	2, 3	Q9 silent-failure-mode primary defense
Receipt-vs-alert reconciliation (auto-disable on mismatch)	2, 3	Global auto-disable on first incident — scope TBD
Daily Phase 2 synthetic smoke test	3	Independent of cross-source reconciliation
Append-only SQLite audit log per Phase 2 purchase	2, 3, 4	Photo perceptual hashes + Telegram tap event + receipt
Wallapop manual re-auth UX (no silent automated re-login)	4	<code>hardware-hunter login wallapop</code> CLI + cookie persistence
Wishlist YAML validator + dry-run search	4	CLI commands for the operator hat

Capability area	Driven by journeys	Notes
docker-compose install + <code>.env.example</code> + README example wishlists	4, 5	Single launch artifact for OSS forkers
<code>CONTRIBUTING.md</code> + <code>ROADMAP.md</code> as launch artifacts	5	Wishlist examples / prompt improvements / selector patches
Legal disclaimer + secondary-account recommendation in README	5	Customer-FAQ position made operational

Out of scope (explicitly): web UI, multi-tenant deployment, RBAC, cloud-hosted SaaS, third-party API consumers, billing.

Domain-Specific Requirements

The product domain (`general` consumer/homelab tooling) has no regulatory regime equivalent to HIPAA/PCI/etc. — but six cross-cutting constraints, locked through five PRFAQ stages, shape every requirement that follows. They get a home here so they cannot drift implicit.

Compliance & Legal Posture

- **Wallapop ToS (rev. Apr 2026):** explicitly forbids scraping/bots. Realistic enforcement is account ban, not legal action (precedent: Tatuck/wallapop-scraper publicly hosted on GitHub; ZebraBot operating commercially in Spain for years without legal escalation).
- **Operating posture:** comply-don't-fight on any C&D. Solo maintainer cannot litigate; pretending otherwise is unsustainable. README/repo position hardware-hunter as a "personal monitoring tool," not a "Wallapop scraper" — wording matters.
- **C&D contingency (documented in `ROADMAP.md` as a possible end state, not a goal):** read carefully, do not respond same-day, assess scope, comply with reasonable scope (rename, archive, code removal). MIT-licensed forks survive.
- **Spanish web-scraping precedent:** Spanish courts have generally permitted scraping of public data (cited generically in customer FAQ, no specific case citation in shipped docs).
- **Hacienda / tax integration: out of scope.** Personal use only by (c3); commercial-volume use would be a (c3) violation.

Privacy & Data Handling (RGPD posture)

- **All data local.** SQLite store on the user's homelab box; no remote logging, no telemetry, no cloud sync.
- **User = data controller; hardware-hunter = processor.** Sellers' publicly posted listing data is processed only for the duration of evaluation/transaction. No profiling, no third-party sharing.
- **Stored data classes:** wishlist YAML (user content), seen-listings dedup index (URL + perceptual photo hash), Phase 2 audit log (alert + tap + transaction), Wallapop session cookie file. All

readable by the user via the filesystem.

- **Retention:** indefinite local retention by default; user owns deletion. No automatic purge requirements (no remote sync to comply with).
- **Cookie/credential hygiene:** restrictive filesystem permissions on cookie file and `.env`; never logged.

Phase 2 Dispute Evidence

Three artifacts persisted **locally, append-only**, per Phase 2 purchase. Defends against three distinct claim classes:

Artifact	Contents	Defends against
Alert snapshot	Listing URL, title, description, photo perceptual hashes, price, wishlist entry matched, LLM evaluation + confidence, Phase 2 settings active at the time	"You bought the wrong thing" (vs seller — preserved even if seller edits/deletes the listing)
User tap event	Telegram message ID, button pressed (Buy / Skip / Snooze), timestamp	"I never authorized this" (Telegram's own audit trail is a corroborating second source)
Marketplace transaction	Receipt ID (Wallapop Pay reference, eBay order number), price actually paid, screenshot of confirmation page (TinyFish Browser captures)	Platform dispute resolution (marketplace logs primary; local trail corroborates if contested)

Anti-Bot Mitigation

- **Polling at human-volume rates.** Cron-driven, not high-frequency streaming. Concrete rate limits captured in NFRs.
- **Wallapop long-lived sessions with manual re-auth.** No silent automated re-login (anti-bot risk). Operator runs `hardware-hunter login wallapop` on cookie expiry.
- **Stealth Chromium for Phase 2 buy flow.** Via TinyFish Browser. Login flows go through real browser sessions, not API token forgery.
- **Per-purchase circuit breaker.** N consecutive Phase 2 buy failures auto-disables autonomous mode globally; protects against silent anti-bot detection escalation.
- **No mass-scraping pattern.** Listing fetches are scoped to wishlist matches, not bulk extraction. Reduces both anti-bot signal and LPI database-rights exposure.

LLM Bias Surfaces

Per the ACES (Columbia/Yale, WebConf 2026) findings on VLM shopping agents, structurally-eliminated biases vs unaudited residual surfaces:

Eliminated by wishlist-anchored design:

- Position bias (each listing evaluated in isolation, no "first result wins")

- Sponsored-content bias (Wallapop and eBay.es do not have sponsored-listing slots equivalent to ACES marketplaces)
- Open-ended price sensitivity (ceilings come from user YAML, not LLM judgment)

Unaudited residual surfaces (must surface in NFRs and the launch-blocker checklist):

- Photo-quality bias (cleaner photos → higher confidence?)
- Description-length bias (verbose → “trustworthy”?)
- Language-register bias (Castilian vs Catalan/regional Spanish/Basque) — matters specifically for a Spain tool; could systematically affect non-Castilian sellers
- Confidence-level calibration (over/under-confidence in particular conditions)

Mitigation path. Empirical bias audit before Phase 2 enables for any user beyond ifuensan; community accuracy dashboard as the long-term audit instrument. Until the dashboard exists, residual bias is best-effort and disclosed in [README.md](#).

Repository Hygiene (legal-driven)

- No Wallapop trademarks, logos, or proprietary terms in repo name, package names, domain, or anywhere user-visible.
- README positions hardware-hunter as a personal monitoring tool, NOT a “Wallapop scraper.” Wording is operational, not aesthetic — reduces the trademark complaint surface and the cost of a forced rename.
- Adapter file names use marketplace names only where strictly necessary (e.g. [wallapop_adapter.py](#) is fine inside [adapters/](#); the package itself is not named after Wallapop).
- [CONTRIBUTING.md](#) carries an explicit “no arbitrage PRs” rule and a pointer to the future-research separate-repo path; closes the door on (c3)-violating contributions before they land.
- [ROADMAP.md](#) names: future-multi-marketplace, future-arbitrage-as-separate-repo, “C&D-induced sunset” as documented end state.

***Risk register cross-reference.** All domain-driven risks (legal, ToS, privacy, anti-bot, bias) are tracked alongside technical and resource risks in the consolidated tables under [Project Scoping & Phased Development > Risk Mitigation Strategy](#).*

Innovation & Novel Patterns

Detected Innovation Areas

Four innovations, ordered by structural significance:

1. Wishlist-anchored LLM evaluation (the framing innovation)

Standard VLM shopping-agent pattern (ACES: Columbia/Yale, WebConf 2026): user gives an open-ended preference (“find me a cheap NAS”); agent picks. Documented to suffer from position bias,

sponsored-content bias, and open-ended price sensitivity.

hardware-hunter inverts it. The user's YAML names exact references and per-SKU price ceilings; the LLM is asked one question per listing — *"does this listing match this wishlist entry?"* — and returns a confidence level. The LLM never picks what to buy; it only verifies.

This is a deliberate framing choice with three structural consequences:

- ACES position/sponsored/pricing biases are eliminated by construction, not by mitigation.
- Arbitrage drift is structurally prevented — the schema has no `expected_resale_value` field, the prompt has no resale framing, the codepath has no "interesting" off-wishlist surfacing. Drift requires a deliberate change visible in code review.
- The remaining LLM failure surface is one verifiable question per listing, orthogonal to the buy-flow guardrails that defend Phase 2.

2. Container detection (the headline differentiator)

A second per-SKU price ceiling — `max_price_in_device` — paired with `container_keywords[]` lets the agent flag listings where the wanted SKU is hidden inside a wrapper (HDD inside a NAS, RAM inside a mini-PC, GPU inside a workstation). The LLM evaluates title/description/photos for whether the wanted part is present at the secondary ceiling.

No surveyed OSS Wallapop bot (wallabot, Walla-Bot, daverbot, nadiamoe, Tatuck) does this. Wallapop's saved searches catch 0% by definition.

The novelty is the *schema commitment*, not the implementation: making "this part hidden in something else" a first-class wishlist concept rather than a heuristic on top of keyword search.

3. Adapter discipline as a v1 launch blocker

`PageFetcher` and `BrowserSession` (or equivalent) interfaces wrap Hermes and TinyFish. Direct imports from business logic = launch blocker, verified via CI lint or code-review checklist. Stack swap (TinyFish → Playwright self-hosted, Hermes pin to a known-good version, etc.) becomes days of work, not weeks.

Pattern is well-known in production engineering; the innovation is **enforcing it as a launch blocker on a single-developer evening project**. The PRFAQ verdict makes this binding: the project's stack-risk story collapses without it.

4. Phase 2 self-disabling guardrail stack (the silent-failure defense)

Three independent defenses against the Q9 scenario (wrong-but-internally-consistent values bypassing structural guardrails):

- **Cross-source price reconciliation at buy time.** Re-fetch via the alternate path (TinyFish if API was used, API if TinyFish was used); disagreement beyond tolerance fails closed.
- **Receipt-vs-alert reconciliation.** After every Phase 2 purchase, diff alert price vs marketplace receipt. Mismatch raises high-priority Telegram and auto-disables Phase 2.
- **Daily synthetic smoke test.** Independent fetch against a known-price synthetic listing. Drift auto-disables Phase 2.

Each catches a different failure shape; the smoke test alone misses bugs whose triggering data is absent from the synthetic fixtures (Journey 3 demonstrates this — comma-formatted prices), and reconciliation alone is silent if the agent never buys. The stack works because the defenses are independent.

Pattern is borrowed from production SRE (independent observability, defense in depth). Innovation is the application: an OSS personal-use shopping agent applying production-grade silent-failure defenses because real money is on the line per purchase.

Market Context & Competitive Landscape

Surveyed OSS Wallapop bots and adjacent tools (May 2026):

Tool	LLM	Container detection	Phase 2 / autonomous purchase	eBay.es	Notes
wallabot	No	No	No	No	Selenium + Telegram alerts; seen-ads.txt dedup
Walla-Bot (miqueasmd)	No	No	No	No	Config-driven keyword search; email alerts
davertor/wallapop-scraper	No	No	No	No	Library returning DataFrames; no integration
nadiamoe/wallabot	No	No	No	No	Alert engine on keyword search
Tatuck/wallapop-scraper	Gemini 2.0 Flash for resale value	No	No	No	Closest in spirit; arbitrage-framed (different product)
Apify Wallapop Scraper	No	No	No	No	SaaS, \$0.50/1000 results
ScrapingBee	No	No	No	N/A	Generic scraping API
Wallapop saved searches	N/A	No (0% by definition)	No	N/A	Native, free, the minimum viable alternative
ZebraBot (zebrabot.es)	Unknown (commercial)	Unknown	Unknown	Unknown	Commercial Spanish service, decade+ live, not OSS — useful as ToS-enforcement precedent

Tool	LLM	Container detection	Phase 2 / autonomous purchase	eBay.es	Notes
hardware-hunter	Wishlist-anchored, confidence-leveled	Yes	Yes (opt-in, HITL Telegram, per-entry)	Yes (official API)	The combination is the moat

The four-bullet differentiator — wishlist YAML with per-SKU ceilings + container detection + opt-in Phase 2 via protected payment rails + eBay.es alongside Wallapop — is the combination none of the surveyed tools offers. Tatuck is the closest single-axis competitor (LLM evaluation) but framed as arbitrage; the framing difference is the moat.

Validation Approach

Each innovation has a concrete validation path:

Innovation	How we know it works	How we know it failed
Wishlist-anchored evaluation	First 30 days: ifuensan accepts $\geq 80\%$ of fired alerts as true positives on tap (rough sanity bar; tune with experience). LLM bias audit before Phase 2 enables for non-Castilian/non-self users.	If precision drops below $\sim 50\%$, the prompt or the model is wrong; reframe before Phase 2.
Container detection	First 90 days: ≥ 1 user-confirmed true-positive container alert. PRFAQ-bound success metric.	Zero true positives across 90 days = headline differentiator is vapor; rethink before public framing.
Adapter discipline	CI lint enforces no direct Hermes/TinyFish imports from business-logic packages. Stack swap dry run (mock the alternate adapter) succeeds in a single evening.	A direct import slips into main = process failure; tighten the lint and treat as launch blocker.
Phase 2 guardrail stack	First 90 days of Phase 2: zero silent mispurchases. Smoke-test regression set grows with each marketplace UI surprise.	Any silent mispurchase in the first 90 days = stack insufficient; pause Phase 2, root-cause the gap, add a fourth defense if needed.

Pre-launch: the accuracy dashboard (community-collected) is committed in PRFAQ as a launch-week priority, not a fast-follow. It is the long-term validation instrument for both wishlist-anchored evaluation and container detection.

Risk Mitigation (innovation-specific fallbacks)

- **Wishlist-anchored evaluation underperforms.** Fallback: more verbose `keywords[]` per entry (looser surface) + raise the confidence-threshold gate so only high-confidence matches alert; sacrifices recall for precision.
- **Container detection produces too many false positives.** Fallback: tighten `container_keywords[]`, raise the confidence threshold for container matches specifically (per-entry-class threshold, not

just per-entry). Acceptable degradation: alerts on wrappers go quiet; direct-listing alerts continue to work.

- **Adapter discipline slips during a rushed patch.** Fallback: CI lint hard-fails the build; a direct import never reaches main. Process is the safety net.
- **Phase 2 guardrail stack misses a silent failure.** Fallback: per-purchase circuit breaker is the universal backstop — N consecutive Phase 2 anomalies (any kind) auto-disable autonomous mode globally. Adding a fourth independent defense is cheaper than waiting for the next failure class.
- **All four innovations underperform together.** Fallback: hardware-hunter degrades to a more sophisticated wallabot — keyword alerts on Wallapop + eBay.es with deduplication. Still beats saved searches (eBay.es coverage, dedup, two-tier pricing) and hits the time-tax success criterion. The product survives even if the headline novelty doesn't.

CLI Tool Specific Requirements

Project-Type Overview

hardware-hunter has two interaction surfaces:

- **Daemon-style operation** (the steady state). Once installed and configured, hardware-hunter runs as a long-lived agent process inside docker-compose. **Hermes Agent's built-in scheduler** drives all polling jobs (no external cron). The daemon never blocks on user input; alerts go to Telegram; failures auto-disable rather than escalate.
- **Operator CLI** (the maintenance surface). A single `hardware-hunter` binary with subcommands handles install-time setup, manual re-auth, wishlist editing, dry-runs, and audit-log inspection. `ifuensan-as-operator` (Journey 4) is the primary CLI consumer.

The Telegram bot is the primary *user* surface (alerts, taps, decisions). The CLI is the primary *operator* surface (setup, troubleshooting, audit). The two surfaces never overlap — no operator command takes Telegram input, and no Telegram interaction triggers CLI work.

Technical Architecture Considerations

- **Scheduler:** Hermes Agent's native scheduler (per kickoff: natural-language cron, no job limit, persists across restarts via Hermes' SQLite memory). hardware-hunter expresses polling cadence as Hermes scheduled jobs, not OS cron. Stack-swap implication: if Hermes is ever replaced, a `Scheduler` interface must abstract this — captured under adapter discipline.
- **Two-path Wallapop adapter** behind `PageFetcher` interface (unofficial API primary + TinyFish Search/Fetch fallback).
- **eBay.es adapter** behind same `PageFetcher` interface (official eBay API primary; no fallback needed at v1 — eBay's API is structurally stable).
- **Phase 2 buy flow** behind `BrowserSession` interface (TinyFish Browser via MCP).

- **LLM evaluation** behind a `ListingEvaluator` interface (provider-agnostic; Gemini Flash assumed for cost; pluggable).
- **All persistence is local SQLite** behind a `Store` interface (seen-listings dedup, audit log, optional cached LLM evaluations).
- **No remote logging, no telemetry.**

Adapter discipline is a v1 launch blocker per the Innovation section; CI lint enforces no direct imports of Hermes / TinyFish / Gemini SDKs from anywhere outside the corresponding adapter package.

Command Structure

A single `hardware-hunter` binary, subcommand-organized. Daemon mode is the implicit default when no subcommand is given (or when launched via docker-compose); subcommands are for the operator.

```
hardware-hunter                # daemon mode (Phase 1 + opted-in Phase 2 entries)
hardware-hunter --version
hardware-hunter --help

# Setup & authentication
hardware-hunter init           # scaffold wishlist.yaml, config.yaml, .env from example
hardware-hunter login wallapop # interactive: opens stealth Chromium, persists cookies
hardware-hunter login ebay     # OAuth flow for eBay.es API

# Wishlist & config
hardware-hunter validate-wishlist [path]      # schema + reachability + duplicate-ref
hardware-hunter validate-config [path]        # config.yaml + .env presence
hardware-hunter test-search <query>|—entry NAME> # dry-run search (no alert sent)
hardware-hunter explain <listing-url>         # one-shot LLM evaluation against current

# Phase 2 controls
hardware-hunter phase2 status                 # show per-entry Phase 2 settings
hardware-hunter phase2 enable <entry>         # turn on for a wishlist entry
hardware-hunter phase2 disable <entry>|—all>  # turn off (per-entry or globally)
hardware-hunter phase2 smoke-test             # run synthetic price-parse smoke test r
hardware-hunter phase2 reconcile <receipt-id> # re-run receipt-vs-alert reconciliation

# Audit & diagnostics
hardware-hunter audit show [--last N | --entry E] # paginated audit log
hardware-hunter audit export [--since DATE] <path> # export audit rows (JSONL)
hardware-hunter health                            # adapter status, scheduler status, last
hardware-hunter logs [--tail | --since DURATION] # structured log access

# Lifecycle
hardware-hunter daemon                          # explicit daemon mode (default; rarely
hardware-hunter stop                            # graceful shutdown signal
```

Shell completion (bash + zsh) is a post-launch nice-to-have, not a v1 requirement. Subcommand grouping keeps completion straightforward to add later.

Output Formats

Surface	Format	Notes
Telegram alert (Phase 1)	Markdown message with photo attachment + inline buttons	Bullets: photo, price, seller location, one-line LLM take, confidence level, deeplink. Inline buttons: [👁 Ver] [👤 Skip] [😴 Snooze 24h].
Telegram alert (Phase 2 enabled entry)	Same as Phase 1 + Buy button	Inline buttons: [✅ Buy] [❌ Skip] [👁 Ver]. Buy executes the buy flow on tap.
Telegram operational alert	Plain text, no buttons	⚠ prefix for high-priority (Phase 2 auto-disable, smoke-test drift, reconciliation tripped). ⓘ prefix for informational (session expiry).
CLI default output	Plain text, human-readable, ANSI colors when stdout is a TTY	<code>--no-color</code> to force plain.
CLI structured output	<code>--format json</code> on commands that have a list/object result	<code>test-search</code> , <code>audit show</code> , <code>health</code> , <code>phase2 status</code> , <code>explain</code> . JSON on stdout, errors on stderr.
Logs (daemon)	Structured JSON Lines on stdout	Single line per event; <code>level</code> , <code>ts</code> , <code>event</code> , <code>entry</code> , <code>marketplace</code> , <code>listing_id</code> , <code>latency_ms</code> as standard fields. docker-compose captures stdout.
Audit log (Phase 2)	Append-only SQLite	Tables: <code>alert_snapshots</code> , <code>tap_events</code> , <code>transactions</code> . Photo perceptual hashes stored inline. Exportable to JSONL via <code>audit export</code> .
Seen-listings dedup	SQLite	URL + perceptual photo hash + first-seen timestamp + last-seen timestamp + match-fired flag.

Telegram message format is **fixed for v1** (changes require schema migration in the audit log because alert snapshots reference it). Operator-facing CLI output is allowed to evolve more freely.

Config Schema

Three files, three concerns:

File	Concern	Sensitivity	Tracked in repo?
<code>wishlist.yaml</code>	What to look for	User content	Example version (<code>wishlist.example.yaml</code>) tracked; user's actual file gitignored
<code>config.yaml</code>	Operational tunables	Non-sensitive	Example version (<code>config.example.yaml</code>) tracked; user's <code>config.yaml</code> gitignored
<code>.env</code>	Credentials only	Sensitive	<code>.env.example</code> tracked; <code>.env</code> gitignored, never logged

wishlist.yaml (schema sketch — full schema pinned in Architecture step)

```
- manufacturer: Western Digital
  model: WD Red Plus 4TB
  ref: WD40EFPX
  type: hdd
  max_price_solo: 55          # €
  max_price_in_device: 90    # €; nil disables container detection for this entry
  keywords:
    - "WD Red 4TB"
    - "WD40EFPX"
  container_keywords:
    - "NAS Synology 4TB"
    - "NAS QNAP 4TB"
  phase2:
    enabled: false           # default false; flipped by `phase2 enable <entry>`
    confidence_threshold: high # one of: low, medium, high
```

Constraints: `manufacturer` / `model` / `ref` define the unique key; `type` ∈ {`hdd`, `ram`}; lists kept under ~100 entries (validated by `validate-wishlist`); **no** `expected_resale_value` / `min_margin_percent` / `current_market_price` **fields permitted** — `validate-wishlist` rejects them with a pointer to the (c3) scope contract and the future-research repo path.

config.yaml (operational tunables)

```
schedule:
  wallapop_poll: "every 15 minutes"      # natural language for Hermes scheduler
  ebay_poll:     "every 30 minutes"
  phase2_smoke:  "daily at 06:00"

rate_limits:
  tinyfish_search_rpm: 5
  tinyfish_fetch_rpm:  25
  ebay_api_rpd:        5000

phase2:
  default_enabled: false                # never set true here; per-entry only
  circuit_breaker_threshold: 3          # N consecutive failures → global auto-disable
  reconcile_tolerance_eur: 0.50         # cross-source price reconciliation tolerance
  reconcile_tolerance_pct: 0.5          # whichever is greater of the two
  on_reconciliation_failure: global     # one of: global, per_entry # see Open Question

llm:
  provider: gemini-flash                # one of: gemini-flash, gpt-4o, claude-haiku
  confidence_default: high              # default per-entry confidence threshold

paths:
  data_dir: /var/lib/hardware-hunter    # SQLite, audit log, cookies
  log_dir:  /var/log/hardware-hunter    # rotated by Hermes/docker

logging:
  level: info                           # debug, info, warn, error
  format: json                           # json, text
```

.env (credentials only)

```
# Telegram
TELEGRAM_BOT_TOKEN=
TELEGRAM_CHAT_ID=

# Wallapop (cookie file persisted separately by `hardware-hunter login wallapop`)
WALLAPOP_USERNAME=
WALLAPOP_PASSWORD=

# eBay.es (OAuth tokens persisted separately after `hardware-hunter login ebay`)
EBAY_CLIENT_ID=
EBAY_CLIENT_SECRET=

# TinyFish (MCP)
TINYFISH_API_KEY=

# LLM
LLM_API_KEY=
```

`.env` is loaded once at process start; rotation requires daemon restart. No secret hot-reload at v1.

Scripting Support

- **Idempotency.** All `validate-*`, `test-search`, `explain`, `audit show`, `phase2 status`, `health` commands are read-only and safe to invoke repeatedly. `init` refuses to overwrite existing files (use `--force` with confirmation prompt).
- **Exit codes.**
 - `0` — success
 - `1` — usage error / unknown subcommand
 - `2` — config or wishlist validation failure
 - `3` — adapter / network failure (Wallapop down, eBay API 5xx, TinyFish unreachable)
 - `4` — auth failure (cookie expired, OAuth token invalid)
 - `5` — Phase 2 guardrail tripped (smoke-test drift, reconciliation mismatch, circuit breaker open)
- These codes are stable across releases and documented in README.
- **Pipeable JSON.** Every list/object-returning subcommand supports `--format json`. JSON goes to stdout; errors and progress to stderr. Suitable for `jq` chains in operator scripts.
- **No interactive prompts in non-TTY contexts.** When stdout is not a TTY, commands fail fast on missing args rather than prompting (`hardware-hunter login wallapop` is the explicit exception — it's interactive by design).
- **Daemon lifecycle.** `hardware-hunter daemon` (or implicit default) handles SIGTERM gracefully (drains in-flight LLM evaluations, flushes audit log, lets Telegram alerts complete, exits ≤ 30 s). docker-compose `stop_grace_period: 30s` matches.
- **Dry-run philosophy.** `test-search` and `explain` never mutate state, never send Telegram messages, never count against rate limits beyond the actual fetch. Result printed as if the alert *would* fire — operator sees confidence, message body, button set.

Sections Skipped

Per CSV `skip_sections` for `cli_tool`: `visual_design`, `ux_principles`, `touch_interactions`.

Confirmed correct: hardware-hunter has no visual UI; UX is structured around Telegram conversational patterns covered in the Output Formats subsection above.

Implementation Considerations

- **Hermes Agent integration.** Polling jobs registered with Hermes' scheduler at startup; subagents (up to 8 concurrent per kickoff doc) parallelize per-marketplace and per-listing LLM evaluation. Hermes' `clarify` primitive is *not* used in the daemon path (the daemon never asks the operator at runtime — it Telegrams or fails closed); `clarify` is permissible in the operator CLI for safety prompts (e.g. `phase2 disable --all` confirmation).
- **TinyFish MCP.** Configured in Hermes' MCP server list; `tinyfish_search`, `tinyfish_fetch`, `tinyfish_browser` tools attach automatically. Adapter wrappers normalize their interfaces.

- **Memory / LLM evaluation cache.** Hermes' SQLite memory + FTS5 hosts a per-listing-URL cache of LLM evaluation results — avoids re-querying when the same listing is re-fetched within a TTL. TTL defaults to 24 h; shorter for low-confidence evaluations.
- **Container packaging.** Single `Dockerfile` produces an image that bundles the Python (or chosen runtime) project + Hermes' agent runtime. `docker-compose.yml` mounts `./data` for the SQLite store and `./config` for the three config files. Phase 2 needs no extra container — TinyFish Browser is invoked over MCP, not embedded.
- **Versioning.** Semver. `0.x` until first Phase 2 purchase by ifuensan; `1.0.0` after Phase 1 stabilizes for 4–8 weeks AND a successful Phase 2 purchase has been completed.

Project Scoping & Phased Development

This section provides the strategic context behind the Phase 1 / Phase 2 / Vision split already established in `Product Scope`. No requirements are being deferred or de-scoped here — the phasing is the one the PRFAQ and kickoff documents pre-committed to.

MVP Strategy & Philosophy

MVP approach: *problem-solving MVP* — the Phase 1 release exists to prove that a wishlist-anchored, container-aware, two-marketplace alert agent meaningfully replaces manual scanning for the user (ifuensan personally). Adoption, revenue, and community-formation are explicit non-goals of MVP per (c3).

Phase 1 ships first because:

- It validates the wishlist-anchored design against real listings before any money is on the line.
- It generates the data the Phase 2 reconciliation stack and the accuracy dashboard will need (smoke-test fixtures, perceptual photo hashes, real-world false-positive rates).
- It establishes the “trust window” the user needs before flipping any Phase 2 toggle (4–8 weeks of clean Phase 1 reading, per success criteria).

Phase 2 ships next because:

- The customer-FAQ commits to it and removing it would weaken the product against saved searches (which already cover the alert-only minimum).
- The silent-failure-mode defenses (cross-source price reconciliation, receipt-vs-alert diff, daily smoke test) cannot be fully validated without Phase 1 data; shipping them earlier would be premature.
- The `4–8 weeks of Phase 1 stable` gate is a *behavioral* requirement on the user, not a code-completion gate — Phase 2 code can ship dark behind the per-entry toggle.

Vision items defer because:

- Multi-marketplace expansion was explicitly captured-and-reverted during PRFAQ Stage 2; relitigating it without Wallapop+eBay.es validation would invalidate the (c3) scope contract.

- Arbitrage is a different product entirely, structurally barred from this repo by schema, prompt, and CONTRIBUTING.md guard-rails.
- agentskills.io publication is a community-distribution decision that depends on whether v1 delivers on its commitments.

Resource Requirements

Phase	Effort (single experienced developer, evenings + weekends)	Calendar
Phase 1	~3–5 weeks of focused engineering, +1 week of contingency for Wallapop unofficial-API quirks	~1.5 months elapsed (with day-job hours)
Phase 2	~4–8 additional weeks (PRFAQ Q5: lower bound slips ~50% in practice if Phase 2 buy-flow stability matches Q1’s “hardest problem” ranking)	~2 additional months elapsed
Total	~3 months of solid evening work to ship everything the customer FAQ promised	~3.5 months elapsed total

Team: ifuensan, solo. No co-developers, no funding, no contractor budget.

Skills required (already held): Python (or chosen runtime), agent-runtime familiarity (Hermes), MCP integration, basic SRE practices, Spanish second-hand-marketplace context.

What the project explicitly says no to in v1 (per PRFAQ verdict):

- Any side project consuming the same evening hours.
- Multi-marketplace expansion (deferred per (c3)).
- Polish, dashboards, web UI, accuracy reporting beyond the launch-week dashboard.
- Feature work outside customer-FAQ commitments before Phase 2 ships.

Risk Mitigation Strategy

Technical risks

Risk	Mitigation	Triggered fallback
Phase 2 buy flow stability (PRFAQ-named hardest problem)	Phase 1 stabilization gate (4–8 weeks); fail-closed UI element checks; per-purchase circuit breaker; integration tests against recorded fixtures	Phase 1 stays the steady state if selector breakage outpaces patch cycles. Honest trade-off, not failure.
Silent Phase 2 misbehavior (Q9 scenario)	Three independent defenses: cross-source price reconciliation at buy time, receipt-vs-alert reconciliation, daily synthetic smoke test (covered in Innovation section + Phase 2 Failure Defense FRs)	Per-purchase circuit breaker auto-disables Phase 2 globally on any anomaly; user investigates audit log; manual re-enable after green smoke test.
Wallapop unofficial API breaks	Two-path adapter: API primary + TinyFish Search/Fetch fallback; either path carries Phase 1 alone	TinyFish covers Phase 1 if API path dies entirely; eBay.es independent path also continues.

Risk	Mitigation	Triggered fallback
Wallapop session persistence	Long-lived sessions with manual re-auth (no silent automated re-login); operator runs <code>hardware-hunter login wallapop</code> on expiry	Manual re-auth is acceptable friction (Journey 4); automation here is anti-bot-risky.
Both Wallapop adapter paths fail simultaneously	Two independent paths reduce probability; eBay.es independent adapter path keeps running regardless	Phase 1 degraded on Wallapop only; user notified via operational Telegram; operator patches at next opportunity.
Wallapop ToS account ban	Customer-FAQ recommends secondary account ("willing to lose"); polling at human-volume rates; no mass-scraping; no silent re-login	Re-auth with replacement account; not a project-ending risk for any individual user.
TinyFish service change or pricing	<code>PageFetcher</code> / <code>BrowserSession</code> interfaces; Playwright self-hosted as bare-metal fallback	Adapter swap is days of work, not weeks. Worst-case Phase 1 cost ~€10/month.
Hermes Agent breaking change	Pin to known-good version (<code>v0.13.x</code> floor at <code>v1</code>); MIT license = fork option; primitives used (cron, memory, clarify) are core	Migrate if a single-dev evening absorbs it; otherwise stay pinned, security-only fork-with-patch if needed.
LLM bias on language register	Empirical bias audit before Phase 2 enables for non-Castilian users; community accuracy dashboard as long-term audit instrument	Continue Phase 1 alerts (low-stakes) while audit is pending.

Market / adoption risks

Risk	Mitigation	Triggered fallback
Container-detection accuracy poor	Accuracy dashboard treated as launch-week priority, not fast-follow; container detection framed as "best-effort" pre-launch	Tighten <code>container_keywords[]</code> and confidence thresholds; if precision <50%, container alerts go quiet while direct-listing alerts continue. Product survives.
Wallapop ToS exposure (C&D arrives)	Comply-don't-fight posture; repo hygiene (no Wallapop trademarks); secondary-account recommendation in customer FAQ	C&D-induced sunset documented as ROADMAP end state; rebrand-and-comply procedure. MIT forks survive.
User-perceived "yet another bot"	PRFAQ four-bullet differentiator + customer FAQ explicitly distinguishes from competitors; honest framing tells some readers to use saved searches instead	Acceptable; (c3) means adoption is a side effect anyway.
Low real-world differentiator validation at launch	Accuracy dashboard committed as launch-week priority; pre-launch comms frame container detection as best-effort	If first 90 days show zero true-positive container alerts: rethink headline framing before promoting publicly.

Resource / sustainability risks

Risk	Mitigation	Triggered fallback
Solo-maintainer burnout	Five concrete walk-away triggers documented (personal-use, technical-debt, sustained-burden, legal, stack); graceful off-ramp procedure	Pinning commit + README addendum + repo archived with fork pointer. Not silent abandonment.
Maintenance burden exceeds budget	Steady-state target ≤ 8 h/month; rolling 3-month average > 20 h/month invokes sustained-burden walk-away	If walk-away fires: graceful off-ramp; forks pick up if the codebase has value to others.
OSS contributor pool small	CONTRIBUTING.md names three explicit invitation categories (wishlist examples, prompt improvements, Wallapop selector patches)	(c3): OSS adoption isn't a success criterion. If no contributors, that's fine — same posture as solo from day one.
Stack-component costs increase	TinyFish free tier disappearing $\rightarrow \sim \text{€}10/\text{month}$ worst case; Browser pricing $\rightarrow$ cents-per-purchase. All trivial vs deal value.	Cost-creep audit; switch to self-hosted Playwright if economics flip.
Day-job time competing for evenings	Walk-away triggers explicitly include 3-month personal-disuse and 20+ h/month-for-3-months patterns	Same off-ramp procedure; ifuensan's homelab can also fall back to saved searches without losing parts already bought.

Functional Requirements

*This section is the **capability contract** for hardware-hunter v1. UX, architecture, and epic decomposition must trace every feature back to one or more FRs below. A capability not listed here will not exist in the final product unless explicitly added via PRD revision.*

Wishlist Management

- **FR1.** The user can declare wishlist entries in a YAML file with fields for manufacturer, model, reference, type (`hdd/ram`), maximum standalone price (`max_price_solo`), maximum in-device price (`max_price_in_device`), keywords list, and container keywords list.
- **FR2.** The user can specify a per-entry Phase 2 enable/disable flag and a per-entry confidence threshold (`low/medium/high`).
- **FR3.** The operator can run a wishlist validator that verifies schema conformance, uniqueness of (`manufacturer, model, ref`) keys, soft-cap of ~ 100 entries, and structural absence of arbitrage-related fields (`expected_resale_value`, `min_margin_percent`, `current_market_price`); the agent refuses to load any wishlist containing such fields and points the operator to the (c3) scope contract and the future-research repo path.
- **FR4.** The agent uses (`manufacturer, model, ref`) as the entry key for alerts, audit log, dedup, and Phase 2 controls.

- **FR5.** Setting `max_price_in_device` to nil disables container detection for that entry; the entry continues to alert on direct matches against `max_price_solo`.

Marketplace Monitoring

- **FR6.** The agent monitors Wallapop continuously via two independent paths — a primary unofficial-API path and a fallback search/fetch path — such that either path alone can carry Phase 1 alerts.
- **FR7.** The agent monitors eBay.es continuously via the official eBay API, structurally independent from any Wallapop adapter, so a Wallapop break leaves eBay.es coverage intact.
- **FR8.** The agent polls each marketplace at human-volume rates configurable per marketplace (e.g. Wallapop every 15 minutes, eBay every 30 minutes), driven by Hermes Agent's built-in scheduler.
- **FR9.** The agent generates marketplace-specific search queries from each wishlist entry's `keywords` and `container_keywords`.
- **FR10.** The agent persists a seen-listings dedup index (URL + perceptual photo hash + first-seen and last-seen timestamps + match-fired flag) so a single listing fires at most one alert per wishlist entry.
- **FR11.** The agent never surfaces listings that do not match any wishlist entry, regardless of how interesting they appear; there is no "good deals" surfacing path.
- **FR12.** The agent stops polling Wallapop and emits an operational Telegram alert when the Wallapop session expires; it never attempts silent automated re-login.

Listing Evaluation

- **FR13.** For each candidate listing, the agent invokes a wishlist-anchored LLM evaluation that answers one question — *"does this listing match this wishlist entry?"* — and returns a confidence level (`low/medium/high`).
- **FR14.** The agent evaluates standalone listings against the entry's `max_price_solo` ceiling and container/wrapper listings against `max_price_in_device`, using `container_keywords` to identify wrapper candidates (NAS, mini-PC, workstation, etc.).
- **FR15.** The agent surfaces the LLM's one-line take on listing authenticity (e.g. *"photos show a real WD Red, serial visible"*) in every alert.
- **FR16.** The agent caches LLM evaluation results per listing URL with a configurable TTL (default 24h, shorter for low-confidence results) to avoid redundant queries on re-fetch.
- **FR17.** The agent never scores listings for resale value, margin, expected market value, or any arbitrage-related metric; the LLM has no codepath that produces such outputs.

Alert Notifications (Phase 1)

- **FR18.** The user receives a Telegram alert per matched listing containing photo, price, seller location, one-line LLM take, confidence level, matched wishlist entry, and a deep link to the listing.
- **FR19.** Phase 1 alerts include inline action buttons: *View*, *Skip*, *Snooze*.
- **FR20.** The user can tap *Snooze* on any alert to suppress further alerts for the same wishlist entry for a configurable window (default 24h).

- **FR21.** The agent emits operational Telegram alerts — distinct from listing alerts — for: marketplace authentication expiry, Phase 2 auto-disable events, smoke-test drift, circuit-breaker openings, and reconciliation tripping. Operational alerts are prefixed ( for high-priority,  for informational) and contain no inline action buttons.
- **FR22.** The Telegram alert format for Phase 1 and Phase 2 listing alerts is fixed for v1; changes require a coordinated audit-log schema migration because alert snapshots reference the format.

Autonomous Purchase (Phase 2)

- **FR23.** The user can enable Phase 2 per wishlist entry (default disabled). Phase 2 is never enabled by default for any entry; there is no setting that flips it on globally.
- **FR24.** Phase 2-enabled entry alerts include *Buy*, *Skip*, *View* buttons; tapping *Buy* initiates the autonomous purchase flow.
- **FR25.** The agent completes Phase 2 purchases exclusively via platform-protected payment rails (Wallapop Pay, eBay.es checkout); the agent has no codepath that uses Bizum, transferencia, or any unprotected rail.
- **FR26.** The agent enforces per-entry maximum prices as a hard ceiling; any listing exceeding the ceiling fails closed without offering a *Buy* button, regardless of confidence.
- **FR27.** The agent enforces per-entry confidence thresholds; listings below the threshold present a manual-review-only path even when Phase 2 is enabled for the entry.
- **FR28.** The agent verifies all expected UI elements are present in the marketplace buy flow before proceeding (fail-closed UI check); any missing element aborts the purchase and emits an operational Telegram alert.
- **FR29.** The agent has no fully-autonomous mode. There is no setting, flag, environment variable, or CLI command that bypasses the per-purchase Telegram tap.
- **FR30.** Phase 2 buy flows execute via a stealth browser session (real browser, not API token forgery), using the marketplace's own login state.

Phase 2 Failure Defense

- **FR31.** Before completing a Phase 2 purchase, the agent re-fetches the listing via the alternate marketplace path (cross-source price reconciliation) and aborts the purchase if prices disagree beyond a configurable tolerance (€ floor + percentage, whichever is greater).
- **FR32.** After every Phase 2 purchase, the agent compares the alert price to the marketplace receipt price (receipt-vs-alert reconciliation); a mismatch raises a high-priority Telegram alert and auto-disables Phase 2 (scope of auto-disable — global vs per-entry — captured as Open Question).
- **FR33.** The agent runs a daily synthetic Phase 2 smoke test against a known-price fixture; drift between parsed and independent values auto-disables Phase 2 globally.
- **FR34.** The agent maintains a per-purchase circuit breaker that auto-disables Phase 2 globally after N consecutive Phase 2 failures (default 3, configurable).
- **FR35.** After any Phase 2 auto-disable, the operator must explicitly re-enable Phase 2 via the CLI; the agent never re-enables itself, regardless of subsequent successful smoke tests.

Audit & Dispute Evidence

- **FR36.** The agent persists an append-only SQLite audit log per Phase 2 purchase with three artifacts:
 - **Alert snapshot** — listing URL, title, description, photo perceptual hashes, price, wishlist entry matched, LLM evaluation + confidence level, Phase 2 settings active at the time.
 - **User tap event** — Telegram message ID, button pressed (*Buy / Skip / Snooze*), timestamp.
 - **Marketplace transaction** — receipt ID (Wallapop Pay reference, eBay order number), price actually paid, screenshot of the confirmation page.
- **FR37.** The operator can view audit log entries (`audit show`, optionally scoped by entry or time window) and export them to JSONL (`audit export`).
- **FR38.** All audit log data is stored locally; the agent never transmits audit data to remote servers and emits no telemetry of any kind.

Operator Tools & Configuration

- **FR39.** The operator interacts with hardware-hunter through a single `hardware-hunter` binary with subcommands. Daemon mode is the implicit default when no subcommand is given.
- **FR40.** The operator can scaffold initial config files (`wishlist.yaml`, `config.yaml`, `.env`) from tracked examples via `init`; the command refuses to overwrite existing files unless `--force` is given alongside an interactive confirmation prompt.
- **FR41.** The operator can authenticate Wallapop interactively via `login wallapop`, which opens a real browser session, walks the operator through credentials and 2FA, and persists the resulting cookie with restrictive filesystem permissions.
- **FR42.** The operator can authenticate eBay.es via `login ebay`, completing OAuth and persisting tokens locally with restrictive permissions.
- **FR43.** The operator can perform a dry-run search (`test-search`) against a wishlist entry or arbitrary query without sending alerts, mutating state, or counting beyond actual rate-limit usage.
- **FR44.** The operator can perform a one-shot LLM evaluation of any listing URL (`explain`) to inspect how the agent would treat it, including confidence and the alert message body the agent would have sent.
- **FR45.** The operator can view, enable, and disable per-entry Phase 2 settings via `phase2 status`, `phase2 enable <entry>`, and `phase2 disable <entry|—all>`.
- **FR46.** The operator can manually trigger the synthetic smoke test (`phase2 smoke-test`) and re-run a receipt-vs-alert reconciliation on a past receipt (`phase2 reconcile <receipt-id>`).
- **FR47.** The operator can inspect agent health (`health`) — adapter status, scheduler status, last poll, last alert, last Phase 2 event — to diagnose problems without reading raw logs.
- **FR48.** All read-only operator commands support a `--format json` flag for scripting; daemon logs emit structured JSON Lines on stdout. Operator commands return stable, documented exit codes (0 success / 1 usage / 2 validation / 3 adapter / 4 auth / 5 Phase 2 guardrail).
- **FR49.** Configuration is split across `wishlist.yaml` (user content), `config.yaml` (operational tunables: rates, thresholds, paths, log level), and `.env` (credentials only, never logged); the agent loads `.env` once at process start with no hot-reload.

- **FR50.** The agent handles SIGTERM gracefully — drains in-flight LLM evaluations, flushes the audit log, completes pending Telegram alerts, exits within 30 seconds.

Project Distribution & Artifacts

- **FR51.** The repository ships a single `docker-compose.yml` install path with example wishlist entries for common HDD and RAM models, an `.env.example`, and a `config.example.yaml`; user-specific files (`wishlist.yaml`, `config.yaml`, `.env`) are gitignored.
- **FR52.** The repository includes a `CONTRIBUTING.md` with an explicit “no arbitrage PRs” rule and three named invitation categories (wishlist examples, prompt improvements, Wallapop selector patches), pointing to a separate-repo path for arbitrage forks.
- **FR53.** The repository includes a `ROADMAP.md` naming future-multi-marketplace expansion, future-arbitrage-as-separate-repo, and “C&D-induced sunset” as a documented possible end state.
- **FR54.** The README positions hardware-hunter as a personal monitoring tool (not a “Wallapop scraper”), includes a legal disclaimer covering Spanish ToS posture and the secondary-account recommendation, and contains no Wallapop trademarks, logos, or proprietary terms in titles, package names, or domain references.

Non-Functional Requirements

This section specifies HOW WELL hardware-hunter must perform, not WHAT it does. Categories not listed are explicitly out of scope: **Scalability** (single-user product; forks are independent installs; wishlist bounded at ~100 entries), **Accessibility** (Telegram is the user surface, accessibility delegated to Telegram clients; the CLI is operator-only).

Performance

The performance envelope is shaped by the 4-hour listing half-life and the customer-FAQ commitment that “the manual refresh disappears.”

- **NFR-P1. Alert delivery latency.** Time from listing publication to Telegram alert delivery: ≤ 20 minutes p95 in steady state (bounded by polling cadence + LLM evaluation + Telegram delivery). Wallapop poll cadence (default 15 min) is the dominant component; tunable per `config.yaml`.
- **NFR-P2. Phase 2 buy completion.** End-to-end Phase 2 buy completion (Telegram tap → marketplace receipt screenshot): ≤ 60 seconds p95 under normal marketplace load. Cross-source reconciliation pre-check accounts for ≤ 10 seconds of that budget.
- **NFR-P3. LLM evaluation latency.** Per-listing evaluation: ≤ 5 seconds p95 with Gemini Flash (assumed default). The agent processes evaluations concurrently (Hermes subagents, up to 8 workers) so total per-poll evaluation time scales with batch size, not serial latency.
- **NFR-P4. CLI responsiveness.** Read-only operator commands (`audit show`, `health`, `phase2 status`, `validate-wishlist`, `validate-config`) complete in ≤ 2 seconds for typical wishlist sizes (≤ 100 entries).

- **NFR-P5. Daemon startup.** Cold boot from `docker-compose up` to first scheduled poll registered with Hermes scheduler: ≤ 30 seconds.

Security

- **NFR-S1. Credential handling.** All marketplace credentials, Telegram bot tokens, TinyFish API keys, and LLM API keys are loaded exclusively from `.env` at process start; the agent never logs credentials, never persists them outside the cookie/token files, and never transmits them to remote services beyond their target API.
- **NFR-S2. Cookie file permissions.** Wallapop session cookie file and eBay OAuth token file are created with mode `0600` (owner read/write only). The agent verifies permissions at startup and refuses to load if mode is permissive.
- **NFR-S3. Transport security.** All external API calls (Wallapop, eBay, Telegram, TinyFish, LLM provider) use TLS 1.2 or higher. The agent rejects connections that fall back to weaker protocols or accept invalid certificates (no `--insecure` equivalent).
- **NFR-S4. Audit log integrity.** The Phase 2 audit log is append-only at the application layer (no `UPDATE` or `DELETE` statements ever issued by hardware-hunter against `alert_snapshots`, `tap_events`, or `transactions`). Existing rows are never mutated; corrections, if needed, are appended as new annotation rows referencing the original.
- **NFR-S5. Payment-rail enforcement.** The agent has no codepath that initiates a transfer outside Wallapop Pay or eBay.es checkout. Any code change that adds an alternate rail requires an explicit PRD revision; CI lint flags introductions of relevant API calls outside the protected-rail wrapper.
- **NFR-S6. Operator confirmation for destructive ops.** Operator commands that affect Phase 2 globally (`phase2 disable --all`) or destroy state (`init --force`) require an interactive confirmation prompt; in non-TTY contexts these commands fail rather than auto-proceed.
- **NFR-S7. Local-only data plane.** The agent emits no telemetry, no usage analytics, no crash reports to any external service. Logs go to stdout (captured by docker-compose); audit log stays local; SQLite stores stay local.

Reliability & Recovery

- **NFR-R1. eBay.es independence.** A complete Wallapop outage (both adapter paths down) does not affect eBay.es polling, alerts, or Phase 2 (for eBay.es-source listings). The two marketplace adapters share no runtime state at the polling/evaluation/alerting layer.
- **NFR-R2. Two-path Wallapop fallback.** When the unofficial-API path fails (timeout, 4xx/5xx, schema mismatch), the agent automatically falls back to the TinyFish search/fetch path within the same poll cycle. The agent logs the path used per request for diagnosis.
- **NFR-R3. Graceful degradation, not silent failure.** When a capability is degraded (one Wallapop path down, LLM provider rate-limited, smoke test failing), the agent emits an operational Telegram alert. No degradation is silent.
- **NFR-R4. Manual-recovery boundaries.** The agent **never** attempts the following automatically: silent re-login on Wallapop session expiry (FR12); Phase 2 re-enable after auto-disable (FR35); overwrite of an existing config file (FR40). All require explicit operator action.

- **NFR-R5. Daemon crash behavior.** On unhandled exception, the agent exits non-zero with a structured log line containing the exception class and stack trace; docker-compose `restart: on-failure` (with backoff) is the supported recovery model. The audit log and seen-listings dedup must remain consistent across crash/restart (no duplicate alerts, no missing audit rows).
- **NFR-R6. Marketplace-break recovery target.** After a marketplace UI/API change breaks one adapter, the operator is expected to restore service within ≤ 30 hours of patch effort per break, ≤ 3 attempts. Beyond either threshold, the technical-debt walk-away trigger fires (Project Sustainability success criterion).

Integration

- **NFR-I1. Hermes Agent.** Pinned to a known-good version range (default v0.13.x at v1; floor / ceiling specified in dependency manifest). Migration to a new minor version is treated as a code change requiring CI green and operator-side smoke test before merge.
- **NFR-I2. TinyFish MCP.** Configured as a Hermes MCP server endpoint; the agent does not embed TinyFish SDKs directly. Free-tier rate limits respected by config (5 req/min Search, 25 URLs/min Fetch); the agent enforces these client-side regardless of remote enforcement.
- **NFR-I3. LLM provider abstraction.** A `ListingEvaluator` interface wraps the LLM call; provider switch (Gemini Flash $\rightarrow$ GPT-4o $\rightarrow$ Claude Haiku) requires only an adapter swap and a config change, not business-logic edits. CI lint enforces no direct LLM-SDK imports outside the adapter package.
- **NFR-I4. Wallapop unofficial-API contract drift.** The adapter validates the response schema at parse time; schema drift surfaces as an `adapter` failure (exit code 3 / operational alert), not silent acceptance.
- **NFR-I5. eBay.es official API.** Standard API key + OAuth flow; renewals and rate-limit headers respected. Daily request budget tracked against the configured ceiling (`ebay_api_rpd`); breach raises an operational alert and degrades to reduced poll cadence rather than failing.
- **NFR-I6. Telegram bot delivery.** Failed sends (network error, Telegram 5xx) are retried with exponential backoff up to a configurable ceiling (default 3 attempts over ~ 1 minute). Persistent failure surfaces as a structured-log error; the agent does not block polling on Telegram outages.

Cost

- **NFR-C1. Phase 1 monthly cost target.** $\leq \text{€}0/\text{month}$ on existing homelab hardware; $\leq \text{€}10/\text{month}$ worst case (small VPS + TinyFish free tier disappearance + LLM at ~ 50 entries $\times \sim 10$ candidates/day $\times \sim 500$ tokens/eval). The first fair deal caught typically covers a year of running costs at this envelope.
- **NFR-C2. Phase 2 incremental cost.** Per-purchase cost remains in cents range with TinyFish Browser default pricing; $\leq \text{€}1.00$ per Phase 2 purchase even under worst-case TinyFish pricing changes. Cost-creep audit is a documented operator workflow; threshold breach triggers a config or provider review.
- **NFR-C3. LLM cache hit rate.** The per-listing evaluation cache (FR16) targets a hit rate $\geq 60\%$ on listings re-fetched within TTL during steady state, reducing token spend during dedup re-checks and operator dry-runs.

Maintainability

- **NFR-M1. Adapter discipline (launch blocker).** No business-logic package directly imports Hermes, TinyFish, Wallapop SDK, eBay SDK, LLM SDK, or marketplace-specific HTML/CSS selectors. Direct imports are caught by CI lint (custom import-graph rule) and block merge. This is a v1 launch-blocker NFR per the Innovation section.
- **NFR-M2. Test coverage on Phase 2 critical path.** Phase 2 buy-flow logic (cross-source reconciliation, fail-closed UI checks, circuit breaker, audit-log writes, receipt-vs-alert reconciliation) has integration tests against recorded marketplace fixtures. Coverage on these modules $\geq 90\%$ line coverage at v1.0.
- **NFR-M3. Smoke-test regression set.** The synthetic Phase 2 smoke test (FR33) maintains a regression set that grows with every marketplace UI surprise (parser drift, format change, button rename); fixtures are tracked in the repo, not generated.
- **NFR-M4. Semver discipline.** Public CLI surface (subcommand names, flag names, exit codes, JSON output schema), config schema (`wishlist.yaml`, `config.yaml`, `.env` keys), and audit-log SQLite schema are governed by semver. Breaking changes bump major.
- **NFR-M5. Dependency footprint.** Total Python (or chosen runtime) third-party dependency count kept under 30 direct dependencies at v1; each new direct dependency requires a “why not standard library or existing dep” note in the PR. Hermes, TinyFish, and LLM SDK adapters concentrate the heavyweight deps inside their own packages.
- **NFR-M6. Solo-maintainer sustainability.** Steady-state maintenance budget ≤ 8 h/month after the first 6 months; rolling 3-month average > 20 h/month invokes the sustained-burden walk-away trigger. README addendum + dependency-pinning final commit + repo archival is the documented graceful off-ramp procedure (covered in Project Sustainability success criterion).

Privacy

- **NFR-PR1. Data classes.** The agent stores only: wishlist YAML (user content), config files (operational tunables), credentials (`.env`, cookie file, OAuth token file), seen-listings dedup index (URL + perceptual photo hash + timestamps + match flag), Phase 2 audit log (alert + tap + transaction). No other personal data, no other listing metadata beyond what the listing exposes publicly.
- **NFR-PR2. Retention.** Indefinite local retention by default. The user, as data controller, owns deletion. The agent provides no automated purge of seen-listings or audit data; this is intentional (forensic value of audit log; dedup correctness).
- **NFR-PR3. No remote persistence.** Configuration may not be set in any way that causes seen-listings, audit-log, or wishlist data to be transmitted to remote storage. The architecture forbids it; the schema has no field for it.
- **NFR-PR4. Deletion path.** The operator can delete agent state by removing `data_dir/*` (SQLite stores) and credential files; no cloud-side “right to be forgotten” workflow exists or is needed.
- **NFR-PR5. Listing data scope.** The agent processes seller-published listing data (title, description, photos, price, location) only for the duration of evaluation and audit-snapshot capture. No profiling of sellers, no cross-listing analytics, no aggregation beyond per-listing dedup.

Observability

- **NFR-O1. Structured logs.** Daemon emits structured JSON Lines on stdout with the standard fields `level`, `ts`, `event`, `entry`, `marketplace`, `listing_id`, `latency_ms`, `error_class`. Logs are docker-compose-captured; no syslog/remote-logging integration at v1.
- **NFR-O2. Health surface.** `health` command returns adapter status (Wallapop primary / Wallapop fallback / eBay.es / Telegram / TinyFish / LLM provider), Hermes scheduler status, last-poll timestamp per marketplace, last-alert timestamp, last Phase 2 event, and current Phase 2 enable/disable scope. Suitable for cron-driven external health checks.
- **NFR-O3. Operator-readable audit log.** `audit show` paginates the Phase 2 audit log with human-readable formatting; `audit export` produces JSONL suitable for `jq` or external analysis.
- **NFR-O4. Diagnostic completeness.** For any operational alert (Phase 2 disable, smoke-test drift, reconciliation tripped, circuit-breaker open, session expiry), the agent emits a structured log entry that contains the data necessary to root-cause the event without re-running the failing path. Operator must not need to enable debug logging to diagnose a production incident.
- **NFR-O5. No mandatory log retention by hardware-hunter.** docker-compose / systemd / docker log driver owns log rotation; the agent makes no assumptions about how long logs are kept.

Open Questions

Genuine unknowns and undecided choices that survive PRD lock-in. Each one names the resolution path so it doesn't drift indefinitely. *Already-resolved* kickoff/PRFAQ questions (project name, license, scheduler, config layout, CLI shape, web backend, Wallapop session persistence strategy, two-path adapter design) are deliberately not listed here.

#	Question	Default for v1	Path to resolution	Owner	Blocking?
OQ1	Receipt-vs-alert reconciliation auto-disable scope: global vs per-entry . Currently <code>on_reconciliation_failure: global</code> in <code>config.yaml</code> ; FR32 codifies global default. User flagged option to make per-entry.	Global	Re-evaluate after first 30 days of Phase 2 with real-world reconciliation outcomes; switch to per-entry only if global causes excessive correlated false-disables.	ifuensan	No — global is safe-default
OQ2	Container detection prompt criterion: strict ceiling against <code>max_price_in_device</code> (current design) vs LLM-judged "vale la pena" qualitative criterion (kickoff doc Q7).	Strict ceiling against <code>max_price_in_device</code> , with LLM confidence as secondary gate	Validate via Phase 1 container alerts in first 90 days; compare false-positive/negative rates of strict vs qualitative on real-world fixtures from the regression set.	ifuensan	No — strict is the safer default

#	Question	Default for v1	Path to resolution	Owner	Blocking?
OQ3	Realistic per-purchase TinyFish Browser cost (NFR-C2 says $\leq$ €1.00/purchase; PRFAQ estimates "cents").	$\leq$ €1.00 worst-case per NFR-C2	Measure on first 5 Phase 2 purchases; update NFR-C2 and customer-FAQ cost numbers before Phase 2 docs go public.	ifuensan	Yes — Phase 2 customer-FAQ docs need this measured before public release
OQ4	Wishlist scale assumptions (~50 entries $\times$ ~10 candidates/day $\times$ ~3M tokens/month).	Assumed correct; cost envelope holds even at 2 $\times$	Validate during first month of personal Phase 1 use; update config-yaml example comments and NFR-C1 if reality diverges materially.	ifuensan	No
OQ5	Per-marketplace adapter break frequency (estimated 2–4/year per PRFAQ; could be higher if Wallapop anti-bot tightens).	2–4/year	Track empirically; if frequency exceeds 6/year sustained, consider whether the technical-debt walk-away trigger threshold (3 failed attempts $\times$ 30h) is still calibrated correctly.	ifuensan	No
OQ6	Language-register bias on LLM evaluation (Castilian vs Catalan/regional Spanish/Basque). Bias audit committed before Phase 2 enables for non-Castilian users — outcome empirically unknown.	Best-effort, disclosed in README	Empirical audit on a fixed corpus of Spanish-region listings before Phase 2 enables for any user beyond ifuensan; results feed the accuracy dashboard.	ifuensan + community	Yes — gates Phase 2 enablement for non-Castilian users
OQ7	Phase 2 fallback when TinyFish Browser is unavailable mid-buy (kickoff doc Q5).	Fail closed; emit operational Telegram alert with the listing link so the user can buy manually within their existing session.	Document the manual-fallback path in README's troubleshooting section before Phase 2 ships.	ifuensan	No — graceful default is fine for v1

#	Question	Default for v1	Path to resolution	Owner	Blocking?
OQ8	agentskills.io publication for Hermes ecosystem visibility (kickoff doc Q10).	Deferred to Vision	Decide post-launch based on whether v1 hits its success criteria and whether the Hermes community shows interest in the wishlist-anchored evaluation pattern.	ifuensan	No